

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

ОТЧЕТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ №2
по дисциплине «Математическая логика и теория формальных
языков»
на тему «Регулярные выражения»
Вариант 16

Выполнил студент группы
№5130201/30101

Радионова П. В. _____

Проверил преподаватель

Востров А. В. _____

«____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание задачи	4
1.1 Содержимое титульного листа лабораторной работы	4
1.2 Регулярные выражения и регулярные множества	4
1.2.1 Формальное определение регулярных выражений	5
1.3 Регулярные выражения для представления титульного листа . .	5
1.4 Конечный автомат	7
1.4.1 Недетерминированный конечный автомат	7
1.4.2 Детерминированный конечный автомат	8
1.4.3 Автоматы в данной работе	8
2 Особенности реализации	16
2.1 Структуры данных	16
2.2 Функции для проверки паттернов	17
2.3 Класс DFA	19
2.3.1 Конструктор класса	19
2.3.2 Методы класса	20
2.4 Генерация титульного листа	27
2.4.1 Методы класса	27
3 Результаты работы	32
Заключение	36
Список использованных источников	37

Введение

Цель работы: реализовать программу, которая проверяет введенный текст через реализацию конечного автомата.

Вариант 16 Проверка корректности титульного листа лабораторной или курсовой работы.

Задачи работы

1. Построить регулярное выражение для проверки корректности титульного листа.
2. Построить недетерминированный конечный автомат, проверяющий корректность титульного листа.
3. Детерминировать полученный недетерминированный конечный автомат.
4. Реализовать программу, которая проверяет введенный текст через реализацию конечного автомата.
5. Реализовать функцию случайной генерации верной строки по полученному конечному автомату.

1 Математическое описание задачи

1.1 Содержимое титульного листа лабораторной работы

В данной работе рассматривается титульный лист для лабораторной или курсовой работы или проекта студента Санкт-Петербургского политехнического университета Петра Великого.

Титульный лист содержит:

- «шапку»:
 - «Министерство науки и высшего образования Российской Федерации» первой строкой,
 - полное наименование ВУЗа второй строкой,
 - полное наименование факультета (института),
 - полное наименование кафедры (высшей школы),
 - код и название направления;
- центральную часть:
 - вид работы (лабораторная работа, курсовая работа, курсовой проект),
 - название дисциплины в кавычках,
 - тема работы в кавычках,
 - может содержать вариант работы;
- номер группы, фамилию и инициалы выполнившего работу студента;
- фамилию и инициалы проверившего работу преподавателя;
- дату сдачи работы;
- город, в котором расположено учебное заведение и год написания работы.

1.2 Регулярные выражения и регулярные множества

Регулярные выражения – это формальный язык шаблонов для поиска и выполнения манипуляций с подстроками в тексте; формула, задающая правило поиска подстрок в потоке символов.

Регулярные множества – это бесконечные множества цепочек, построенных из символов конечного словаря по определенным правилам. Регулярные множества также называют *регулярными языками*, так как они являются множествами цепочек, построенными над конечным словарем.

Регулярное выражение показывает, как можно построить регулярное множество цепочек из одноэлементных множеств с использованием трех операций: конкатенация, объединения и итерации.

Пример Регулярное выражение $ab + ba^*$ задает множество:

$$\{ab\} \cup \{b, ba, baa, baaa, \dots\}.$$

1.2.1 Формальное определение регулярных выражений

Класс регулярных выражений над конечным словарем Σ определяется следующим образом:

- $\emptyset, \epsilon$ и любой символ $a \in \Sigma$ – регулярные выражения;
- если R_1 и R_2 – регулярные выражения, то регулярными выражениями будут:
 - **Объединение** ($R_1 + R_2$ или $R_1 | R_2$): представляет множество цепочек, принадлежащих R_1 или R_2 ,
 - **Конкатенация** ($R_1 R_2$): множество цепочек, образованных соединением цепочки из R_1 и цепочки из R_2 ,
 - **Итерация Клини** (R_1^*): множество цепочек, состоящих из нуля или более повторений цепочек из R_1 ,
 - **Усеченная итерация** (R_1^+): операция, аналогичная итерации, но требует минимум одного повторения ($R_1^+ = R_1 R_1^*$).

Итерации ($*$ и $+$) имеют наивысший приоритет, после чего следует конкатенация, затем объединение. Скобки могут изменять порядок выполнения операций.

1.3 Регулярные выражения для представления титульного листа

Для валидации и генерации титульного листа используется составное регулярное выражение, которое выполняет проверку каждой строки введенного титульного листа:

```
r'МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ\n'
r'Федеральное государственное автономное образовательное учрежде-
ние высшего образования "Санкт-Петербургский политехнический уни-
верситет Петра Великого"\n'
r'(:Институт [А-Яа-яЁё\s]+|[А-ЯЁ] [а-яёА-ЯЁ\s]+ институт)\n'
```

r'Высшая школа [а-яёА-ЯЁё\s]+\n'
 r'Направление:\s+\d{2}\.\d{2}\.\d{2}\s+[А-ЯЁ] [а-яёА-ЯЁё\с]+\n'
 r'(:Отчет о выполнении лабораторной работы|Отчет о выполнении курсовой работы|Отчет о выполнении курсового проекта)\n'
 r'по дисциплине "[А-ЯЁ] [а-яёА-ЯЁё\с]*"\n'
 r'на тему "[А-ЯЁ] [а-яёА-ЯЁё\с]*"\n'
 r'(:Вариант\s+\d{1,2}\n)?'
 r'Выполнил студент группы №\d{7}/\d{5}\s+([А-ЯЁ] [а-яё]+(?:-[А-ЯЁ] [а-яё]+)?)\s+[А-ЯЁ]\.\s*[А-ЯЁ]\.\n'
 r'Проверил преподаватель ([А-ЯЁ] [а-яё]+(?:-[А-ЯЁ] [а-яё]+)?)\s+[А-ЯЁ]\.\s*[А-ЯЁ]\.\n'
 r'«\d{1,2}»\s+(?:января|февраля|марта|апреля|мая|июня|июля|августа|сентября|октября|ноября|декабря)\s+\d{4}\s+г\.\n'
 r'Санкт-Петербург,\s+\d{4}\$'

1. Институт [А-Яа-яЁё\с]+|[А-ЯЁ] [а-яёА-ЯЁё\с]* институт:

- Институт [А-Яа-яЁё\с]+ – слово "Институт после которого следует один или несколько символов: заглавные, строчные буквы или пробелы,
- | – оператор "ИЛИ" разделяющий два допустимых варианта,
- [А-ЯЁ] [а-яёА-ЯЁё\с]+ институт – заглавная буква, за которой следует один или более любых кириллических символов или пробелов;
- \n – символ перевода строки;

2. Направление:\s+\d2\.\d2\.\d2\s+[А-ЯЁ] [а-яёА-ЯЁё\с]+\n:

- \d2\.\d2\.\d2 – запись формата dd.dd.dd, где d – любая цифра;

3. Проверил преподаватель [А-ЯЁ] [а-яё]+\s+[А-ЯЁ]\.\s*[А-ЯЁ]\.\n:

- [А-ЯЁ] [а-яё]+\s+[А-ЯЁ]\.\s*[А-ЯЁ]\. – запись фамилии и инициалов, где фамилия является последовательностью из первого заглавного символа и любого количества строчных, а инициалы – заглавные буквы с точкой.

4. «\d{1,2}»\s+(?:января|февраля|марта|апреля|мая|июня|июля|августа|сентября|октября|ноября|декабря)\s+\d4\s+г\.:

- «\d{1,2}» – запись однозначного или двузначного числа в кавычках,
- января|февраля|марта|апреля|мая|июня|июля|августа|сентября|октября|ноября|декабря – наименование одного из месяцев,
- \d4\s+г\. – запись года в формате «dddd г.», где d – любая цифра.

- начальное состояние $s_{01} = \text{begin}$.

Схема НДКА в данной работе приведена на рис. 1.

1.4.2 Детерминированный конечный автомат

Детерминированный конечный автомат (ДКА) в данной работе отличается от НДКА только тем, что в нем появляется одно дополнительное конечное состояние `loose`. Таким образом, множество Σ в ДКА остается аналогичным НДКА, а множество состояний можно описать следующим образом:

$$S_2 = S_1 \cup \{\text{loose}\}.$$

Начальное состояние s_{02} аналогичным образом является состояние `begin`.

1.4.3 Автоматы в данной работе

На основе регулярного выражения был построен недетерминированный конечный автомат, граф которого приведен на рис. 1. затем была выполнена детерминизация имеющегося НДКА, в результате которой был получен детерминированный конечный автомат, граф которого приведен на рис. 2.

В таблице 1 приведены переходы между состояниями в построенном НДКА. Автомат последовательно обрабатывает строки титульного листа, проверяя их на соответствие шаблону. Если все строки соответствуют шаблону, то автомат переходит в финальное состояние `win`.

Переходы в ДКА соответствуют переходам в НДКА, не считая тех, что ведут в конечное состояние `loose`. В таблице 2 приведены переходы из каждого состояния в состояние `loose`.

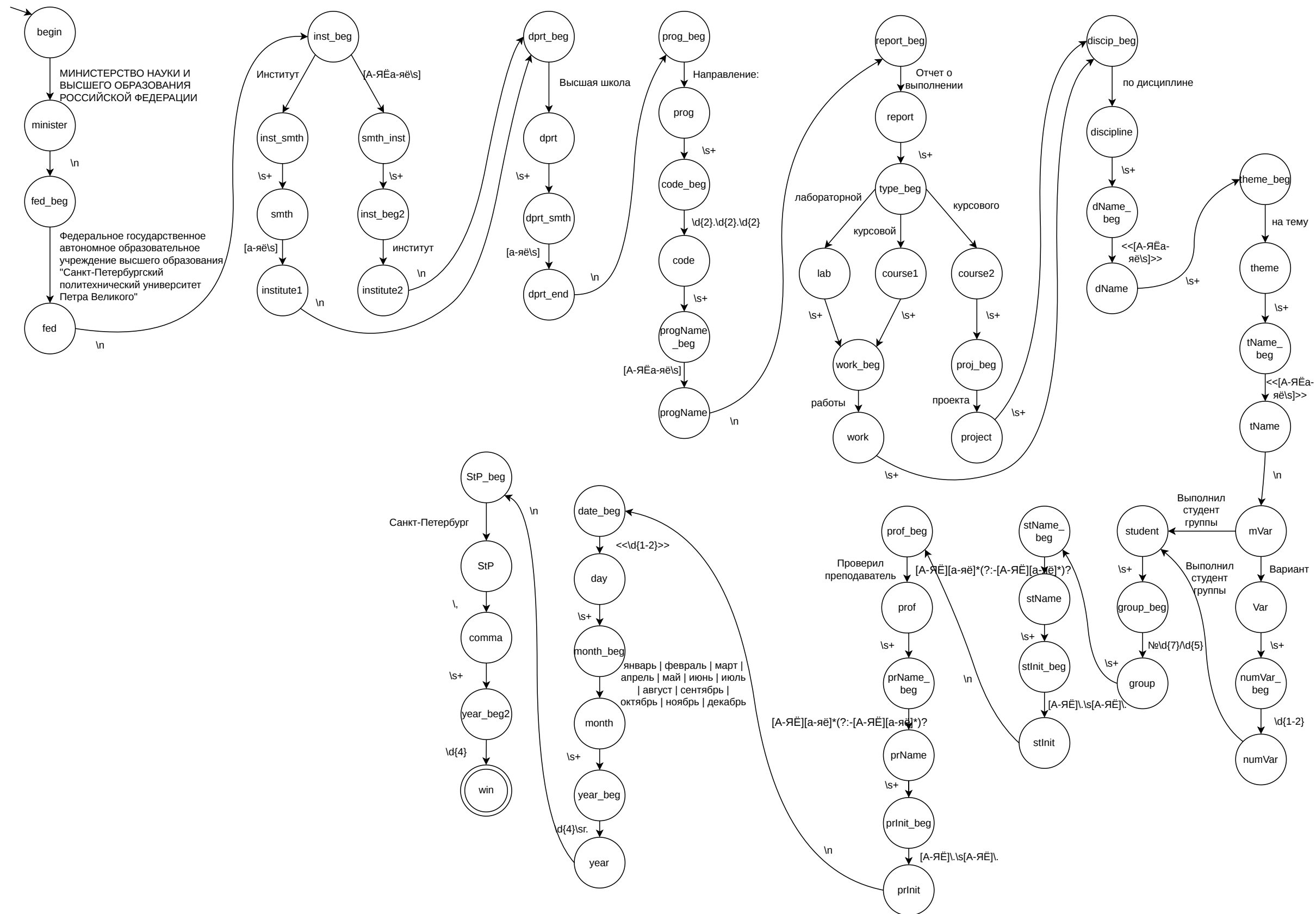


Рис. 1. Недетерминированный конечный автомат

Таблица 1: Таблица переходов НДКА

Текущее состояние	Входной символ	Следующее состояние
begin	МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ	minister
minister	\n	fed_beg
fed_beg	Федеральное государственное автономное образовательное учреждение высшего образования "Санкт-Петербургский политехнический университет Петра Великого"	fed
fed	\n	inst_beg
inst_beg	Институт	inst_smth
inst_beg	[А-ЯЁа-яё\s]	smth_inst
inst_smth	\s+	smth
smth	[а-яё\s]	institutel
smth_inst	\s+	inst_beg2
inst_beg2	институт	institute2
institutel	\n	dprt_beg
institute2	\n	dprt_beg
dprt_beg	Высшая школа	dprt
dprt	\s+	dprt_smth
dprt_smth	[а-яё\s]	dprt_end
dprt_end	\n	prog_beg
prog_beg	Направление:	prog
prog	\s+	code_beg
code_beg	\d{2}.\d{2}.\d{2}	code
code	\s+	progName_beg
progName_beg	[А-ЯЁа-яё\s]	progName
progName	\n	report_beg
report_beg	Отчет о выполнении	report
report	\s+	type_beg
type_beg	лабораторной	lab
type_beg	курсовой	course1
type_beg	курсового	course2
lab	\s+	work_beg
course1	\s+	work_beg
course2	\s+	proj_beg

Текущее состояние	Входной символ	Следующее состояние
work_beg	работы	work
proj_beg	проекта	project
work	\s+	discip_beg
discip_beg	по дисциплине	discipline
discipline	\s+	dName_beg
dName_beg	«[А-ЯЁа-яё\s]»	dName
dName	\s+	theme_beg
theme_beg	на тему	theme
theme	\s+	tName_beg
tName_beg	«[А-ЯЁа-яё\s]»	tName
tName	\n	mVar
mVar	Вариант	Var
mVar	Выполнил студент группы	student
Var	\s+	numVar_beg
numVar_beg	\d{1-2}	numVar
student	\s+	group_beg
group_beg	№\d{7}/\d{5}	group
group	\s+	stName_beg
stName_beg	[А-ЯЁ][а-яё]*(?:-[А-ЯЁ][а-яё]*)?	stName
stName	\s+	stInit_beg
stInit_beg	[А-ЯЁ]\.\s[А-ЯЁ]\.	stInit
stInit	Проверил преподаватель	prof
prof	\s+	prName_beg
prName_beg	[А-ЯЁ][а-яё]*(?:-[А-ЯЁ][а-яё]*)?	prName
prName	\s+	prInit_beg
prInit_beg	[А-ЯЁ]\.\s[А-ЯЁ]\.	prInit
prInit	\n	date_beg
date_beg	«\d{1-2}»	day
day	\s+	month_beg
month_beg	январь февраль март апрель май июнь июль август сентябрь октябрь ноябрь декабрь	month
month	\s+	year_beg
year_beg	\s{4}\sg.	year
year	\n	StP_beg
StP_beg	Санкт-Петербург	StP
StP	,	comma

Текущее состояние	Входной символ	Следующее состояние
comma	\s+	year_beg2
year_beg2	\d{4}	win

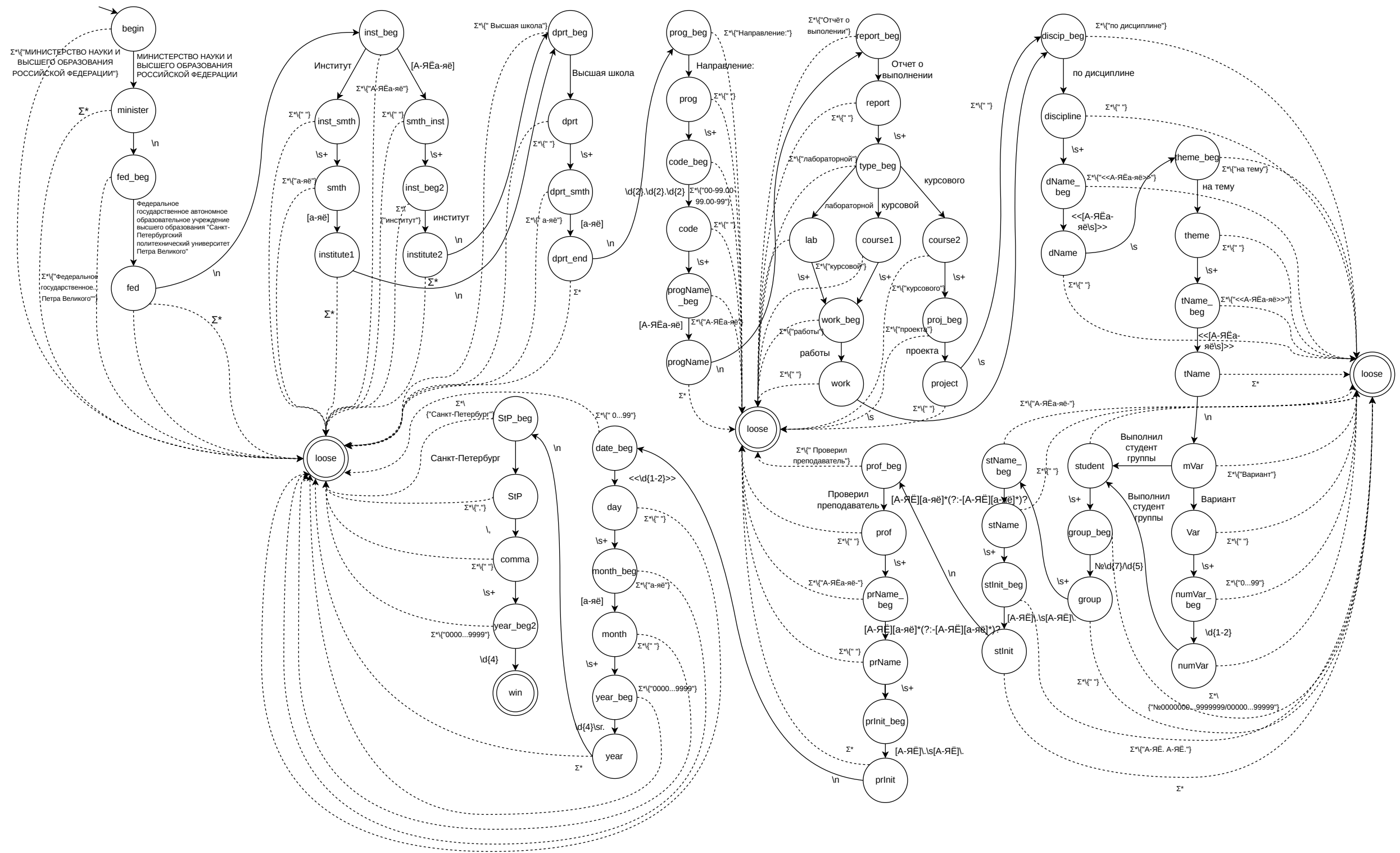


Рис. 2. Детерминированный конечный автомат

Таблица 2: Таблица переходов ДКА в состояние loose

Текущее состояние	Входной символ	Следующее состояние
begin	$\Sigma^* \backslash \text{МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ}$	loose
minister	$\Sigma^* \backslash \backslash n$	loose
fed_beg	$\Sigma^* \backslash \text{Федеральное государственное автономное образовательное учреждение высшего образования "Санкт-Петербургский политехнический университет Петра Великого"}$	loose
fed	$\Sigma^* \backslash \backslash n$	loose
inst_beg	$\Sigma^* \backslash \text{Институт}$	loose
inst_smth	$\Sigma^* \backslash \backslash s +$	loose
smth	$\Sigma^* \backslash [a - \text{яё} \backslash s]$	loose
smth_inst	$\Sigma^* \backslash \backslash s +$	loose
inst_beg2	$\Sigma^* \backslash \text{институт}$	loose
institute1	$\Sigma^* \backslash \backslash n$	loose
institute2	$\Sigma^* \backslash \backslash n$	loose
dprt_beg	$\Sigma^* \backslash \text{Высшая школа}$	loose
dprt	$\Sigma^* \backslash \backslash s +$	loose
dprt_smth	$\Sigma^* \backslash [a - \text{яё} \backslash s]$	loose
dprt_end	$\Sigma^* \backslash \backslash n$	loose
prog_beg	$\Sigma^* \backslash \text{Направление:}$	loose
prog	$\Sigma^* \backslash \backslash s +$	loose
code_beg	$\Sigma^* \backslash \backslash d \{2\} . \backslash d \{2\} . \backslash d \{2\}$	loose
code	$\Sigma^* \backslash \backslash s +$	loose
progName_beg	$\Sigma^* \backslash [A - \text{ЯЁ} a - \text{яё} \backslash s]$	loose
progName	$\Sigma^* \backslash \backslash n$	loose
report_beg	$\Sigma^* \backslash \text{Отчет о выполнении}$	loose
report	$\Sigma^* \backslash \backslash s +$	loose
type_beg	$\Sigma^* \backslash \text{лабораторной}$	loose
lab	$\Sigma^* \backslash \backslash s +$	loose
course1	$\Sigma^* \backslash \backslash s +$	loose
course2	$\Sigma^* \backslash \backslash s +$	loose
work_beg	$\Sigma^* \backslash \text{работы}$	loose
proj_beg	$\Sigma^* \backslash \text{проекта}$	loose
work	$\Sigma^* \backslash \backslash s +$	loose

Текущее состояние	Входной символ	Следующее состояние
discip_beg	$\Sigma^* \backslash \text{по дисциплине}$	loose
discipline	$\Sigma^* \backslash \backslash s +$	loose
dName_beg	$\Sigma^* \backslash \ll [A - \text{Я}\ddot{\text{Е}}a - \text{я}\ddot{\text{е}} \backslash s] \gg$	loose
dName	$\Sigma^* \backslash \backslash s +$	loose
theme_beg	$\Sigma^* \backslash \text{на тему}$	loose
theme	$\Sigma^* \backslash \backslash s +$	loose
tName_beg	$\Sigma^* \backslash \ll [A - \text{Я}\ddot{\text{Е}}a - \text{я}\ddot{\text{е}} \backslash s] \gg$	loose
tName	$\Sigma^* \backslash \backslash n$	loose
mVar	$\Sigma^* \backslash \text{Вариант}$	loose
Var	$\Sigma^* \backslash \backslash s +$	loose
numVar_beg	$\Sigma^* \backslash \backslash d \{1-2\}$	loose
student	$\Sigma^* \backslash \backslash s +$	loose
group_beg	$\Sigma^* \backslash \mathbb{N}^0 \backslash d \{7\} / \backslash d \{5\}$	loose
group	$\Sigma^* \backslash \backslash s +$	loose
stName_beg	$\Sigma^* \backslash [A - \text{Я}\ddot{\text{Е}}][a - \text{я}\ddot{\text{е}}]^* (?: - [A - \text{Я}\ddot{\text{Е}}][a - \text{я}\ddot{\text{е}}]^*)?$	loose
stName	$\Sigma^* \backslash \backslash s +$	loose
stInit_beg	$\Sigma^* \backslash [A - \text{Я}\ddot{\text{Е}}] \backslash . \backslash s [A - \text{Я}\ddot{\text{Е}}] \backslash .$	loose
stInit	$\Sigma^* \backslash \text{Проверил преподаватель}$	loose
prof	$\Sigma^* \backslash \backslash s +$	loose
prName_beg	$\Sigma^* \backslash [A - \text{Я}\ddot{\text{Е}}][a - \text{я}\ddot{\text{е}}]^* (?: - [A - \text{Я}\ddot{\text{Е}}][a - \text{я}\ddot{\text{е}}]^*)?$	loose
prName	$\Sigma^* \backslash \backslash s +$	loose
prInit_beg	$\Sigma^* \backslash [A - \text{Я}\ddot{\text{Е}}] \backslash . \backslash s [A - \text{Я}\ddot{\text{Е}}] \backslash .$	loose
prInit	$\Sigma^* \backslash \backslash n$	loose
date_beg	$\Sigma^* \backslash \ll \backslash d \{1-2\} \gg$	loose
day	$\Sigma^* \backslash \backslash s +$	loose
month_beg	$\Sigma^* \backslash \text{январь} \mid \text{февраль} \mid \text{март} \mid \text{апрель} \mid \text{май} \mid \text{июнь} \mid \text{июль} \mid \text{август} \mid \text{сентябрь} \mid \text{октябрь} \mid \text{ноябрь} \mid \text{декабрь}$	loose
month	$\Sigma^* \backslash \backslash s +$	loose
year_beg	$\Sigma^* \backslash \backslash s \{4\} \backslash \text{sg.}$	loose
year	$\Sigma^* \backslash \backslash n$	loose
StP_beg	$\Sigma^* \backslash \text{Санкт-Петербург}$	loose
StP	$\Sigma^* \backslash ,$	loose
comma	$\Sigma^* \backslash \backslash s +$	loose
year_beg2	$\Sigma^* \backslash \backslash d \{4\}$	loose

2 Особенности реализации

2.1 Структуры данных

В реализации детерминированного конечного автомата (ДКА) проверки корректности титульного листа отчета используются следующие структуры данных:

- `Set[str]`: множество состояний. Хранит все возможные состояния, в которых может находиться автомат в процессе проверки;
- `str = 'begin'`: Определяет точку старта автомата перед обработкой титульного листа;
- `Set[str] = 'win'`: множество допускающих состояний. Определяет состояния, при достижении которых титульный лист считается корректным;
- `Dict[Tuple[str, str], str]`: словарь, где ключ – кортеж (текущее состояние, тип токена), значение – следующее состояние. Реализует функцию переходов автомата;
- `List[str]`: список токенов. Представляет собой промежуточное представление строк титульного листа в виде последовательности лексем;
- `str`: строка, текущее состояние. Хранит состояние автомата в процессе обработки, обновляется на каждом шаге согласно функции переходов.

Дополнительно для токенизации и классификации входных данных используются следующие вспомогательные структуры:

- `ALPHABET`: множество (`set`), содержащее все допустимые символы титульного листа – кириллица (строчные и прописные символы), цифры, пробел, знаки препинания.
- `TOKEN_TYPES`: словарь (`dict[str, str]`), содержащий два фиксированных строковых паттерна, которые должны встречаться в начале любого титульного листа.

Структуры данных

```
1 ALPHABET = set('абвгдеёжзийклмнопрстуфхцщъыьэюя '
2               'АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЩЪЫЬЭЮЯ '
3               '0123456789 .,:;"\'-«»()/\\#№- ' )
4
5 TOKEN_TYPES = {
6     'UPPER_STRICT':
7     ↪ 'МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ',
```



```

7     'FIXED_LINE':
      ↪ 'Федеральное государственное автономное образовательное учреждение'
      ↪ 'высшего образования "Санкт-Петербургский политехнический"'
      ↪ '"университет Петра Великого"',
8 }

```

2.2 Функции для проверки паттернов

Исходное большое регулярное выражение было разбито на несколько регулярных выражений для удобства. Было написано 12 функций для проверки шаблонов выражений титульного листа.

Для каждой функции справедливо следующее.

Вход: строка.

Выход: значение типа bool.

Функция `is_cyrillic_capitalized_phrase` проверяет строку на соответствие шаблону произвольной фразы, которая начинается с прописной буквы и может содержать любое количество строчных и прописных символов и пробелов.

Функция `is_cyrillic_lowercase_phrase` проверяет строку на соответствие шаблону произвольной фразы, которая содержит только строчные буквы и пробелы.

Функции для проверки произвольных фраз

```

1 def is_cyrillic_capitalized_phrase(s: str) -> bool:
2     return bool(re.fullmatch(r'[А-ЯЁ][а-яёА-ЯЁ\s]+', s))
3
4 def is_cyrillic_lowercase_phrase(s: str) -> bool:
5     return bool(re.fullmatch(r'[а-яё\s]+', s))

```

Функция `is_institute_line` проверяет строку на соответствие шаблону наименования института. Такая строка должна содержать подстроку «Институт» в начале, либо «институт» в конце.

Функция `is_hschool_line` проверяет строку на соответствие шаблону наименования высшей школы. Такая строка содержит «Высшая школа» в начале, после чего следует любая фраза на кириллице.

Функция `is_direction_format` проверяет строку на соответствие шаблону направления. Строка должна соответствовать формату «Направление: dd.dd.dd Фраза», где d – любая цифра, а фраза обязательно начинается с заглавной буквы.

Функции для проверки «шапки» титульного листа

```

1 def is_institute_line(s: str) -> bool:
2     if re.match(r'^Институт\s+', s):
3         rest = s[8:].strip()
4         if len(rest) > 0 and re.fullmatch(r'[а-яёА-ЯЁ\s]+', rest):
5             return True
6     if re.search(r'институт$', s):

```

```

7         rest = s[:-8].strip()
8         if len(rest) > 0 and re.fullmatch(r'[А-ЯЁ][а-яёА-ЯЁ\s]*', rest):
9             return True
10        return False
11
12    def is_hschooll_line(s: str) -> bool:
13        if s.startswith('Высшая школа'):
14            rest = s[13:].strip()
15            if len(rest) > 0:
16                if re.fullmatch(r'[а-яёА-ЯЁ\s]*', rest):
17                    return True
18            return False
19
20    def is_direction_format(s: str) -> bool:
21        m = re.match(r'Направление:\s+(\d{2}\.\d{2}\.\d{2})\s+(.+)$', s)
22        if m:
23            rest = m.group(2).strip()
24            if len(rest) > 0 and re.fullmatch(r'[А-ЯЁ][а-яёА-ЯЁ\s]*', rest):
25                return True
26        return False

```

Следующие функции проверяют корректность центральной части титульного листа. Функция `is_report_type` проверяет строку на соответствие одному из трех возможных типов работ (лабораторная работа, курсовая работа, курсовой проект).

Функция `is_quoted_title` проверяет строку на соответствие шаблону фразы с префиксом, после которого следует фраза в кавычках и с прописного символа.

Функция `is_variant` проверяет строку на соответствие шаблону варианта работы, который может быть либо однозначным, либо двузначным числом.

_____ Функции для проверки центральной части титульного листа _____

```

1    def is_report_type(s: str) -> bool:
2        patterns = [
3            r'Отчет о выполнении лабораторной работы$',
4            r'Отчет о выполнении курсовой работы$',
5            r'Отчет о выполнении курсового проекта$'
6        ]
7        return any(re.fullmatch(p, s) for p in patterns)
8
9    def is_quoted_title(s: str, prefix: str) -> bool:
10        pattern = rf'{re.escape(prefix)}\s+"([~"]+)"$'
11        m = re.match(pattern, s)
12        if m:
13            content = m.group(1).strip()
14            if re.fullmatch(r'[А-ЯЁ][а-яёА-ЯЁ\s]*', content):
15                return True
16        return False
17
18    def is_variant(s: str) -> bool:
19        return bool(re.fullmatch(r'Вариант\s+\d{1,2}$', s))

```

Функция `is_student_line` проверяет строку с номером группы, фамилией и инициалами выполнившего работу студента. Номер группы состоит из 7 и 5 символов, разделенных символом «/».

Функция `is_teacher_line` проверяет строку с фамилией и инициалами преподавателя. Фамилия должна начинаться с прописной буквы, остальные символы должны быть строчными буквами. Инициалы должны являться прописными буквами, после которых следует точка.

Функция `is_date_sign` проверяет корректность даты. Число является однозначным или двузначным числом в кавычках, месяц соответствует одному из названий месяца в родительном падеже, год – четырехзначное число с «г.» в конце.

Функция `is_city_year` проверяет последнюю строку титульного листа, которая должна содержать название города «Санкт-Петербург» и год написания работы (четырёхзначное число), разделенные запятой.

Функции для проверки нижней части титульного листа

```
1 def is_student_line(s: str) -> bool:
2     pattern =
3     ↪ r'Выполнил студент группы №(\d{7})/(\d{5})\s+([А-ЯЁ] [а-яё]+(?:-[А-ЯЁ] '
4     ↪ '[а-яё]+)?)\s+([А-ЯЁ])\.\s*([А-ЯЁ])\.$'
5
6     return bool(re.match(pattern, s))
7
8
9
10 def is_teacher_line(s: str) -> bool:
11     pattern =
12     ↪ r'Проверил преподаватель ([А-ЯЁ] [а-яё]+(?:-[А-ЯЁ] [а-яё]+)?)\s+([А-ЯЁ] '
13     ↪ ')\.\s*([А-ЯЁ])\.$'
14
15     return bool(re.match(pattern, s))
16
17
18
19 def is_date_sign(s: str) -> bool:
20     months = ['января', 'февраля', 'марта', 'апреля', 'мая', 'июня',
21               'июля', 'августа', 'сентября', 'октября', 'ноября', 'декабря']
22     months_pattern = '|'.join(months)
23     pattern = rf'«(\d{{1,2}})»\s+({months_pattern})\s+(\d{{4}})\s+r\.$'
24     return bool(re.match(pattern, s))
25
26
27
28 def is_city_year(s: str) -> bool:
29     pattern = r'Санкт-Петербург,\s+(\d{4})$'
30     return bool(re.match(pattern, s))
```

2.3 Класс DFA

Класс DFA реализует логику работы детерминированного конечного автомата, проверяющего корректность титульного листа.

2.3.1 Конструктор класса

Вход: ссылка на создаваемый экземпляр (`self`).

Выход: объект класса DFA с инициализированными состояниями, начальным состоянием, допускающими состояниями, пустым словарём переходов и текущим состоянием, установленным в начальное.

Конструктор класса DFA

```

1 def __init__(self):
2     def __init__(self):
3         self.states: Set[str] = {
4             'begin', 'minister', 'fed_beg', 'fed', 'inst_beg', 'inst_smth',
5             ↪ 'smth',
6             'smth_inst', 'inst_beg2', 'institute1', 'institute2', 'dprt_beg',
7             ↪ 'dprt',
8             'dprt_smth', 'dprt_end', 'prog_beg', 'prog', 'code_beg', 'code',
9             'progName_beg', 'progName', 'report_beg', 'report', 'type_beg',
10            ↪ 'lab',
11            'course1', 'course2', 'work_beg', 'proj_beg', 'work', 'project',
12            'discip_beg', 'discipline', 'dName_beg', 'dName', 'theme_beg',
13            ↪ 'theme',
14            'tName_beg', 'tName', 'mVar', 'Var', 'numVar_beg', 'numVar',
15            ↪ 'student',
16            'group_beg', 'group', 'stName_beg', 'stName', 'stInit_beg',
17            ↪ 'stInit',
18            'prof_beg', 'prof', 'prName_beg', 'prName', 'prInit_beg',
19            ↪ 'prInit',
20            'date_beg', 'day', 'month_beg', 'month', 'year_beg', 'year',
21            'StP_beg', 'StP', 'comma', 'year_beg2', 'win', 'loose'
22        }
23
24        self.start_state: str = 'begin'
25        self.accept_states: Set[str] = {'win'}
26        self.delta: Dict[Tuple[str, str], str] = {}
27        self.current_state: str = self.start_state
28
29        self._build_transitions()

```

2.3.2 Методы класса

Метод `_build_transitions`

Вход: экземпляр класса DFA.

Выход: модифицированный словарь `self.delta`.

Функция задает правила перехода между состояниями на основе типов токенов и заполняет словарь `self.delta`.

Функция `_build_transitions`

```

1 def _build_transitions(self):
2     self.delta[('begin', 'UPPER_STRICT')] = 'minister'
3     self.delta[('minister', 'NEWLINE')] = 'fed_beg'
4     self.delta[('fed_beg', 'FIXED_LINE')] = 'fed'
5     self.delta[('fed', 'NEWLINE')] = 'inst_beg'
6
7     self.delta[('inst_beg', 'INSTITUT_KEYWORD')] = 'inst_smth'
8     self.delta[('inst_smth', 'LOWERCASE_PHRASE')] = 'institute1'

```

```

9     self.delta[('institute1', 'NEWLINE')] = 'dprt_beg'
10
11     self.delta[('inst_beg', 'CAPITALIZED_PHRASE')] = 'smth_inst'
12     self.delta[('smth_inst', 'INSTITUT_KEYWORD_LOWER')] = 'institute2'
13     self.delta[('institute2', 'NEWLINE')] = 'dprt_beg'
14
15     self.delta[('dprt_beg', 'HSCHOOL_KEYWORD')] = 'dprt'
16     self.delta[('dprt', 'LOWERCASE_PHRASE')] = 'dprt_end'
17     self.delta[('dprt_end', 'NEWLINE')] = 'prog_beg'
18
19     self.delta[('prog_beg', 'DIRECTION_KEYWORD')] = 'prog'
20     self.delta[('prog', 'CODE_PATTERN')] = 'code'
21     self.delta[('code', 'CAPITALIZED_PHRASE')] = 'progName'
22     self.delta[('progName', 'NEWLINE')] = 'report_beg'
23
24     self.delta[('report_beg', 'REPORT_KEYWORD')] = 'report'
25     self.delta[('report', 'LAB_KEYWORD')] = 'lab'
26     self.delta[('report', 'COURSE1_KEYWORD')] = 'course1'
27     self.delta[('report', 'COURSE2_KEYWORD')] = 'course2'
28
29     self.delta[('lab', 'WORK_KEYWORD')] = 'work'
30     self.delta[('course1', 'WORK_KEYWORD')] = 'work'
31     self.delta[('course2', 'PROJECT_KEYWORD')] = 'project'
32
33     self.delta[('work', 'DISCIPLINE_KEYWORD')] = 'discip_beg'
34     self.delta[('project', 'DISCIPLINE_KEYWORD')] = 'discip_beg'
35     self.delta[('work', 'NEWLINE')] = 'discip_beg'
36     self.delta[('project', 'NEWLINE')] = 'discip_beg'
37
38     self.delta[('discip_beg', 'DISCIPLINE_KEYWORD')] = 'discipline'
39     self.delta[('discipline', 'QUOTED_TITLE')] = 'dName'
40     self.delta[('dName', 'THEME_KEYWORD')] = 'theme_beg'
41     self.delta[('dName', 'NEWLINE')] = 'theme_beg'
42
43     self.delta[('theme_beg', 'THEME_KEYWORD')] = 'theme'
44     self.delta[('theme', 'QUOTED_TITLE')] = 'tName'
45     self.delta[('tName', 'NEWLINE')] = 'mVar'
46
47     self.delta[('mVar', 'VARIANT_KEYWORD')] = 'Var'
48     self.delta[('Var', 'NUMBER_PATTERN')] = 'numVar'
49     self.delta[('numVar', 'STUDENT_KEYWORD')] = 'student'
50     self.delta[('numVar', 'NEWLINE')] = 'numVar'
51     self.delta[('mVar', 'STUDENT_KEYWORD')] = 'student'
52
53     self.delta[('student', 'GROUP_PATTERN')] = 'group'
54     self.delta[('group', 'LASTNAME_PATTERN')] = 'stName'
55     self.delta[('stName', 'INITIALS_PATTERN')] = 'stInit'
56     self.delta[('stInit', 'NEWLINE')] = 'prof_beg'
57
58     self.delta[('prof_beg', 'PROF_KEYWORD')] = 'prof'
59     self.delta[('prof', 'LASTNAME_PATTERN')] = 'prName'
60     self.delta[('prName', 'INITIALS_PATTERN')] = 'prInit'
61     self.delta[('prInit', 'NEWLINE')] = 'date_beg'

```

```

62 self.delta[('date_beg', 'DATE_PATTERN')] = 'day'
63 self.delta[('day', 'MONTH_PATTERN')] = 'month'
64 self.delta[('month', 'YEAR_PATTERN')] = 'year'
65 self.delta[('year', 'NEWLINE')] = 'StP_beg'
66
67
68 self.delta[('StP_beg', 'CITY_KEYWORD')] = 'StP'
69 self.delta[('StP', 'COMMA')] = 'comma'
70 self.delta[('comma', 'YEAR_PATTERN_SIMPLE')] = 'win'
71
72 all_states = [s for s in self.states if s not in ['loose', 'win']]
73 for state in all_states:
74     self.delta[(state, 'ANY_OTHER')] = 'loose'

```

Метод `_tokenize_line`

Вход: экземпляр класса DFA, строка.

Выход: список токенов `List[str]`.

Метод разбивает исходную строку с текстом титульного листа на последовательность токенов (лексем) для последующей обработки автоматом.

```

_____ Метод _tokenize_line _____
1 def _tokenize_line(self, line: str) -> List[str]:
2     tokens = []
3     line = line.strip()
4
5     key_phrases = [
6         'МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ',
7         'Федеральное государственное автономное образовательное учреждение'
8         ↪ 'высшего образования "Санкт-Петербургский политехнический'
9         ↪ 'университет Петра Великого"',
10    ]
11
12    for phrase in key_phrases:
13        if line == phrase:
14            tokens.append(phrase)
15            return tokens
16
17    if line.startswith('Институт '):
18        tokens.append('Институт')
19        rest = line[8:].strip()
20        if rest:
21            tokens.append(rest)
22        return tokens
23
24    if line.endswith(' институт'):
25        rest = line[:-8].strip()
26        tokens.append(rest)
27        tokens.append('институт')
28        return tokens
29
30    if line.startswith('Вышая школа '):
31        tokens.append('Вышая школа')

```

```

30     rest = line[13:].strip()
31     if rest:
32         tokens.append(rest)
33     return tokens
34
35     if line.startswith('Направление:'):
36         match = re.match(r'Направление:\s+(\d{2}\.\d{2}\.\d{2})\s+(.+)$',
37             ↪ line)
38         if match:
39             tokens.append('Направление:')
40             tokens.append(match.group(1))
41             tokens.append(match.group(2))
42             return tokens
43
44     report_patterns = [
45         'Отчет о выполнении лабораторной работы',
46         'Отчет о выполнении курсовой работы',
47         'Отчет о выполнении курсового проекта'
48     ]
49     for pattern in report_patterns:
50         if line == pattern:
51             tokens.append('Отчет о выполнении')
52             if 'лабораторной' in pattern:
53                 tokens.append('лабораторной')
54                 tokens.append('работы')
55             elif 'курсовой' in pattern:
56                 tokens.append('курсовой')
57                 tokens.append('работы')
58             elif 'курсового' in pattern:
59                 tokens.append('курсового')
60                 tokens.append('проекта')
61             return tokens
62
63     quoted_match = re.match(r'(по дисциплине|на тему)\s+"([~"]+)"$', line)
64     if quoted_match:
65         tokens.append(quoted_match.group(1))
66         tokens.append(f'"{quoted_match.group(2)}"')
67         return tokens
68
69     variant_match = re.match(r'Вариант\s+(\d{1,2})$', line)
70     if variant_match:
71         tokens.append('Вариант')
72         tokens.append(variant_match.group(1))
73         return tokens
74
75     student_match =
76     ↪ re.match(r'Выполнил студент группы №(\d{7}/\d{5})\s+([А-ЯЁ] [а-яё]+)'
77         ↪ '\s+([А-ЯЁ]\.\s*[А-ЯЁ]\.)$', line)
78     if student_match:
79         tokens.append('Выполнил студент группы')
80         tokens.append(f"№{student_match.group(1)}")
81         tokens.append(student_match.group(2))
82         tokens.append(student_match.group(3))

```

```

81         return tokens
82
83     teacher_match =
84         ↪ re.match(r'Проверил преподаватель\s+([А-ЯЁ] [а-яё]+)\s+([А-ЯЁ]\.\s*'
85         ↪ '[А-ЯЁ]\.)$', line)
86     if teacher_match:
87         tokens.append('Проверил преподаватель')
88         tokens.append(teacher_match.group(1))
89         tokens.append(teacher_match.group(2))
90         return tokens
91
92     date_match = re.match(r'«(\d{1,2})»\s+([а-яё]+)\s+(\d{4})\s+r\.$', line)
93     if date_match:
94         tokens.append(f"«{date_match.group(1)}»")
95         tokens.append(date_match.group(2))
96         tokens.append(f"{date_match.group(3)} г.")
97         return tokens
98
99     city_match = re.match(r'(Санкт-Петербург),\s+(\d{4})$', line)
100     if city_match:
101         tokens.append('Санкт-Петербург')
102         tokens.append(',')
103         tokens.append(city_match.group(2))
104         return tokens
105
106     tokens.append(line)
107     return tokens

```

Метод `_get_token_type`

Вход: экземпляр класса DFA, строка `str`.

Выход: строка `str`.

Функция определяет, к какому типу токена относится данная строка, последовательно проверяя её через все функции проверки паттернов.

```

1  def _get_token_type(self, token: str) -> str:
2      if token == '\n':
3          return 'NEWLINE'
4
5      if token ==
6          ↪ 'МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ':
7          return 'UPPER_STRICT'
8
9      if token ==
10         ↪ 'Федеральное государственное автономное образовательное учреждение'
11         ↪ 'высшего образования "Санкт-Петербургский политехнический'
12         ↪ 'университет Петра Великого"':
13         return 'FIXED_LINE'
14
15     if token == 'Институт':
16         return 'INSTITUT_KEYWORD'
17
18     if token == 'институт':
19         return 'INSTITUT_KEYWORD_LOWER'
20
21     if token == 'Вышая школа':

```



```

14         return 'HSCHOOL_KEYWORD'
15     if token == 'Направление: ':
16         return 'DIRECTION_KEYWORD'
17     if token == 'Отчет о выполнении':
18         return 'REPORT_KEYWORD'
19     if token == 'лабораторной':
20         return 'LAB_KEYWORD'
21     if token == 'курсовой':
22         return 'COURSE1_KEYWORD'
23     if token == 'курсового':
24         return 'COURSE2_KEYWORD'
25     if token == 'работы':
26         return 'WORK_KEYWORD'
27     if token == 'проекта':
28         return 'PROJECT_KEYWORD'
29     if token == 'по дисциплине':
30         return 'DISCIPLINE_KEYWORD'
31     if token == 'на тему':
32         return 'THEME_KEYWORD'
33     if token == 'Вариант':
34         return 'VARIANT_KEYWORD'
35     if token == 'Выполнил студент группы':
36         return 'STUDENT_KEYWORD'
37     if token == 'Проверил преподаватель':
38         return 'PROF_KEYWORD'
39     if token == 'Санкт-Петербург':
40         return 'CITY_KEYWORD'
41     if token == ', ':
42         return 'COMMA'
43
44     if re.fullmatch(r'№\d{7}/\d{5}', token):
45         return 'GROUP_PATTERN'
46     if re.fullmatch(r'[А-ЯЁ]\.[А-ЯЁ]\.', token):
47         return 'INITIALS_PATTERN'
48     if re.fullmatch(r'[А-ЯЁ][а-яё]+', token):
49         return 'LASTNAME_PATTERN'
50     if re.fullmatch(r'[А-ЯЁ][а-яёА-ЯЁ\s]+', token):
51         return 'CAPITALIZED_PHRASE'
52     if re.fullmatch(r'января|февраля|марта|апреля|мая|июня|июля|августа'
53     ↪ ' | сентября|октября|ноября|декабря', token):
54         return 'MONTH_PATTERN'
55     if re.fullmatch(r'[а-яё\s]+', token):
56         return 'LOWERCASE_PHRASE'
57     if re.fullmatch(r'\d{2}\.\d{2}\.\d{2}', token):
58         return 'CODE_PATTERN'
59     if re.fullmatch(r'\d{1,2}', token):
60         return 'NUMBER_PATTERN'
61     if re.fullmatch(r'«\d{1,2}»', token):
62         return 'DATE_PATTERN'
63     if re.fullmatch(r'\d{4}\s*r\.', token):
64         return 'YEAR_PATTERN'
65     if re.fullmatch(r'\d{4}', token):
66         return 'YEAR_PATTERN_SIMPLE'

```

```

66     if re.fullmatch(r'"[^"]+"', token):
67         return 'QUOTED_TITLE'
68
69     return 'ANY_OTHER'

```

Метод reset

Вход: экземпляр класса DFA.

Выход: экземпляр класса со сброшенным текущим состоянием в начальное.

Метод сбрасывает текущее состояние автомата в начальное для повторного использования.

Метод reset

```

1 def reset(self):
2     self.current_state = self.start_state

```

Метод process

Вход: экземпляр класса DFA, `List[str]` – список строк титульного листа.

Выход: `bool` – соответствует ли последовательность ожидаемому формату.

Основной метод проверки титульного листа. Обрабатывает входной список строк и определяет, соответствует ли он ожидаемому формату.

Метод process

```

1 def process(self, lines: List[str]) -> bool:
2     self.reset()
3
4     full_text = []
5     for i, line in enumerate(lines):
6         if line.strip():
7             tokens = self._tokenize_line(line)
8             full_text.extend(tokens)
9         if i < len(lines) - 1:
10             full_text.append('\n')
11
12     for token in full_text:
13         token_type = self._get_token_type(token)
14
15         key = (self.current_state, token_type)
16         if key in self.delta:
17             self.current_state = self.delta[key]
18         else:
19             fallback_key = (self.current_state, 'ANY_OTHER')
20             if fallback_key in self.delta:
21                 self.current_state = self.delta[fallback_key]
22             else:
23                 return False
24
25     if self.current_state == 'loose':

```

```

26         return False
27
28     return self.current_state in self.accept_states

```

2.4 Генерация титульного листа

Класс `RandomTitleGenerator` предназначен для автоматической генерации синтаксически корректных титульных листов.

2.4.1 Методы класса

Все методы класса являются статическими (`@staticmethod`) и не требуют создания экземпляра класса.

Метод `random_cyrillic_word`

Вход: минимальная и максимальная длина слова, флаг `capitalized` (нужно ли начинать с заглавной буквы).

Выход: сгенерированное слово (`str`).

Осуществляет генерацию случайного слова на кириллице.

Метод `random_cyrillic_word`

```

1 @staticmethod
2 def random_cyrillic_word(min_len=4, max_len=12, capitalized=True):
3     letters = 'абвгдеёжзийклмнопрстуфхцчшщъыьэюя'
4     word = ''.join(random.choice(letters) for _ in
5                     ↪ range(random.randint(min_len, max_len)))
6     if capitalized:
7         word = word.capitalize()
8     return word

```

Метод `random_cyrillic_phrase`

Вход: минимальное и максимальное количество слов, флаг `capitalized` для первого слова.

Выход: сгенерированная фраза (`str`).

Осуществляет генерацию случайной фразы из нескольких слов.

Метод `random_cyrillic_phrase`

```

1 @staticmethod
2 def random_cyrillic_phrase(min_words=2, max_words=5, capitalized=True):
3     words_count = random.randint(min_words, max_words)
4     words = []
5     for i in range(words_count):
6         if i == 0 and capitalized:
7             words.append(RandomTitleGenerator.random_cyrillic_word(4, 10,
8                             ↪ True))
9         else:
10            words.append(RandomTitleGenerator.random_cyrillic_word(4, 10,
11                            ↪ False))
12    return ' '.join(words)

```

Метод `random_institute`

Вход: метод не принимает аргументов.

Выход: строка вида «Институт фраза» или «Фраза институт».

Осуществляет генерацию случайного названия института (один из двух возможных форматов).

Метод `random_institute`

```
1 @staticmethod
2 def random_institute():
3     if random.choice([True, False]):
4         return f"Институт {RandomTitleGenerator.random_cyrillic_phrase(2, 4,
5             False)}"
6     else:
7         return f"{RandomTitleGenerator.random_cyrillic_phrase(1, 4, True)}
8             институт"
```

Метод `random_hschool`

Вход: метод не принимает аргументов.

Выход: строка вида «Высшая школа фраза».

Осуществляет генерацию случайного названия высшей школы.

Метод `random_hschool`

```
1 @staticmethod
2 def random_hschool():
3     return f"Высшая школа {RandomTitleGenerator.random_cyrillic_phrase(2, 4,
4         False)}"
```

Метод `random_direction`

Вход: метод не принимает аргументов.

Выход: строка вида «Направление: dd.dd.dd Фраза».

Осуществляет генерацию случайной строки направления подготовки.

Метод `random_direction`

```
1 @staticmethod
2 def random_direction():
3     code = f"{random.randint(1, 99):02d}.{random.randint(1, 99):02d}.
4     {random.randint(1, 99):02d}"
5     name = RandomTitleGenerator.random_cyrillic_phrase(1, 5, True)
6     return f"Направление: {code} {name}"
```

Метод `random_report`

Вход: метод не принимает аргументов.

Выход: одна из трёх строк: «Отчет о выполнении лабораторной работы», «Отчет о выполнении курсовой работы» или «Отчет о выполнении курсового проекта».

Случайно выбирает тип отчёта.

```

@staticmethod
def random_report():
    return random.choice([
        "Отчет о выполнении лабораторной работы",
        "Отчет о выполнении курсовой работы",
        "Отчет о выполнении курсового проекта"
    ])

```

Осуществляет генерацию строки с фразой в кавычках.

```
@staticmethod
def random_quoted(prefix):
    title = RandomTitleGenerator.random_cyrillic_phrase(1, 4, True)
    return f'{prefix} "{title}"'
```

Осуществляет генерацию строки варианта работы.

```
@staticmethod
def random_variant():
    return f"Вариант {random.randint(1, 99)}"
```

Осуществляет генерацию строки с информацией о студенте.

```
@staticmethod
def random_student():
    group = f"{random.randint(1000000, 9999999)}/{random.randint(10000, 99999)}"
    lastname = RandomTitleGenerator.random_cyrillic_word(5, 12, True)
    initial1 = random.choice('АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ')
    initial2 = random.choice('АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ')
    return f"Выполнил студент группы №{group} {lastname} {initial1}. {initial2}."
```

Осуществляет генерацию строки с информацией о преподавателе.

Метод random_teacher

```
1 @staticmethod
2 def random_teacher():
3     lastname = RandomTitleGenerator.random_cyrillic_word(5, 12, True)
4     initial1 = random.choice('АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ')
5     initial2 = random.choice('АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ')
6     return f"Проверил преподаватель {lastname} {initial1}.{initial2}."
```

Метод random_date_sign

Вход: метод не принимает аргументов.

Выход: строка вида «ДД» месяц ГГГГ г.».

Осуществляет генерацию строки даты подписания.

Метод random_date_sign

```
1 @staticmethod
2 def random_date_sign():
3     day = random.randint(1, 31)
4     month = random.choice(['января', 'февраля', 'марта', 'апреля', 'мая',
5     ↪ 'июня', 'июля', 'августа', 'сентября', 'октября', 'ноября',
6     ↪ 'декабря'])
7     year = random.randint(2020, 2026)
8     return f"«{day}» {month} {year} г."
```

Метод random_city_year

Вход: метод не принимает аргументов.

Выход: строка вида «Санкт-Петербург, ГГГГ».

Осуществляет генерацию заключительной строки с городом и годом.

Метод random_city_year

```
1 @staticmethod
2 def random_city_year():
3     year = random.randint(2020, 2026)
4     return f"Санкт-Петербург, {year}"
```

Метод generate

Вход: флаг include_variant — включать ли строку с вариантом (по умолчанию True).

Выход: многострочная строка (str), представляющая собой корректный титульный лист, готовый для проверки автоматом.

Осуществляет генерацию полного корректного титульного листа.

Метод generate

```
1 @staticmethod
2 def generate(include_variant=True):
3     lines = [
4         "МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ",
5         'Федеральное государственное автономное образовательное'
6         ↪ 'учреждение высшего образования'
7         ↪ '"Санкт-Петербургский политехнический университет Петра Вели-'
8         ↪ 'кого"',
```

```
6         RandomTitleGenerator.random_institute(),
7         RandomTitleGenerator.random_hschool(),
8         RandomTitleGenerator.random_direction(),
9         RandomTitleGenerator.random_report(),
10        RandomTitleGenerator.random_quoted("по дисциплине"),
11        RandomTitleGenerator.random_quoted("на тему"),
12    ]
13
14    if include_variant:
15        lines.append(RandomTitleGenerator.random_variant())
16
17    lines.extend([
18        RandomTitleGenerator.random_student(),
19        RandomTitleGenerator.random_teacher(),
20        RandomTitleGenerator.random_date_sign(),
21        RandomTitleGenerator.random_city_year()
22    ])
23    return "\n".join(lines)
```

3 Результаты работы

На рис. 3 приведен пример работы программы с генерацией титульного листа и его проверки. Программа генерирует только корректные листы, которые всегда проходят проверку автомата.

```
=====
Проверка титульного листа
=====

Меню:
1. Ввести текст для проверки
2. Сгенерировать случайную верную строку
3. Выход
Выберите действие: 2

Сгенерированный титульный лист:

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования "Санкт-Петербургский политехнический университет Петра Великого"
Институт аяакзхр ящдч ьуц
Высшая школа рфкмёне згро цмцвгьлэ
Направление: 75.21.13 Ссзбэфть септцёё дшгеп яьшё
Отчет о выполнении лабораторной работы
по дисциплине "Отмойшма фсьзшмнся йлялг"
на тему "Длхихьфцаж щбяфьёдб эвмьуь нчёц"
Выполнил студент группы №5564709/73954 Мдфддечкпп Ф. Б.
Проверил преподаватель Цьцннфт Ч. Д.
«13» августа 2022 г.
Санкт-Петербург, 2022

Результат: строка соответствует
```

Рис. 3: Результат проверки сгенерированного титульного листа

Для демонстрации ручного ввода титульного листа будем использовать следующий титульный лист, соответствующий шаблону.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования "Санкт-Петербургский политехнический университет
Петра Великого"
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 03.02.01 Математика и компьютерные науки
Отчет о выполнении лабораторной работы
по дисциплине "Математическая логика и теория формальных языков"
на тему "Регулярные выражения"
Вариант 16
Выполнил студент группы №5130201/30101 Радионова П. В.
Проверил преподаватель Востров А. В.
«5» мая 2026 г.
Санкт-Петербург, 2026

На рис. 4 приведен результат проверки титульного листа.


```
Введите строки титульного листа (для завершения ввода введите пустую строку):  
МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
Федеральное государственное автономное образовательное учреждение высшего обра  
Институт компьютерных наук и кибербезопасности  
Высшая школа технологий искусственного интеллекта  
Направление: 03.02.01 Математика и компьютерные науки  
Отчет о выполнении лабораторной работы  
по дисциплине "Математическая логика и теория формальных языков"  
на тему "Регулярные выражения"  
Вариант 16  
Выполнил студент группы №5130201/30101 Радионова П. В.  
Проверил преподаватель Востров А. В.  
«5» мая 2026 г.  
Санкт-Петербург, 2026  
  
Результат: строка соответствует
```

Рис. 4: Результат проверки корректного титульного листа

Для проверки обработки некорректного титульного листа в код направления добавлена буква, из номера группы и года написания работы убраны несколько символов.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования "Санкт-Петербургский политехнический университет
Петра Великого"
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 03.02.016 Математика и компьютерные науки
Отчет о выполнении лабораторной работы
по дисциплине "Математическая логика и теория формальных языков"
на тему "Регулярные выражения"
Вариант 16
Выполнил студент группы №5130201/3010 Радионова П. В.
Проверил преподаватель Востров А. В.
«5» мая 2026 г.
Санкт-Петербург, 202

Результат проверки некорректного титульного листа приведен на рис. 5.

```
Введите строки титульного листа (для завершения ввода введите пустую строку):  
МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
Федеральное государственное автономное образовательное учреждение высшего обр  
Институт компьютерных наук и кибербезопасности  
Высшая школа технологий искусственного интеллекта  
Направление: 03.02.016 Математика и компьютерные науки  
Отчет о выполнении лабораторной работы  
по дисциплине "Математическая логика и теория формальных языков"  
на тему "Регулярные выражения"  
Вариант 16  
Выполнил студент группы №5130201/3010 Радионова П. В.  
Проверил преподаватель Востров А. В.  
«5» мая 2026 г.  
Санкт-Петербург, 202  
  
Результат: строка не соответствует
```

Рис. 5: Результат проверки некорректного титульного листа

Для демонстрации обработки недопустимых символов в год написания работы добавлен знак «\$»:

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования "Санкт-Петербургский политехнический университет
Петра Великого"
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 03.02.016 Математика и компьютерные науки
Отчет о выполнении лабораторной работы
по дисциплине "Математическая логика и теория формальных языков"
на тему "Регулярные выражения"
Вариант 16
Выполнил студент группы №5130201/3010 Радионова П. В.
Проверил преподаватель Востров А. В.
«5» мая 2026 г.
Санкт-Петербург, 202\$

Результат приведен на рис. 6.

Обработка пустой строки приведена на рис. 7.

Обработка некорректного пользовательского ввода приведена на рис. 8.

```
Введите строки титульного листа (для завершения ввода введите пустую строку):  
МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
Федеральное государственное автономное образовательное учреждение высшего обра  
Институт компьютерных наук и кибербезопасности  
Высшая школа технологий искусственного интеллекта  
Направление: 03.02.016 Математика и компьютерные науки  
Отчет о выполнении лабораторной работы  
по дисциплине "Математическая логика и теория формальных языков"  
на тему "Регулярные выражения"  
Вариант 16  
Выполнил студент группы №5130201/3010 Радионова П. В.  
Проверил преподаватель Востров А. В.  
«5» мая 2026 г.  
Санкт-Петербург, 202$  
  
Результат: Титульный лист содержит недопустимые символы
```

Рис. 6: Результат проверки титульного листа с недопустимыми символами

```
Меню:  
1. Ввести текст для проверки  
2. Сгенерировать случайную верную строку  
3. Выход  
Выберите действие: 1  
  
Введите строки титульного листа (для завершения ввода введите пустую строку):  
  
Результат: Пустая строка! Строка не соответствует
```

Рис. 7: Обработка пустой строки

```
Меню:  
1. Ввести текст для проверки  
2. Сгенерировать случайную верную строку  
3. Выход  
Выберите действие: r  
Неверный выбор, попробуйте снова.
```

Рис. 8: Обработка некорректного пользовательского ввода

Заключение

В результате выполнения лабораторной работы было составлено регулярное выражение для обработки титульного листа отчета о выполнении лабораторной работе. На основе этого регулярного выражения был построен недетерминированный конечный автомат, который был детерминирован для получения детерминированного конечного автомата. Также реализована программа на языке Python, которая выполняет проверку строки на соответствие автомату и обеспечивает генерацию корректных титульных листов для проверки.

Достоинства

1. Программа генерирует реалистичные даты.
2. Автомат обеспечивает возможность не добавлять вариант работы, если такового нет.

Недостатки

1. Автомат не предполагает использование форм женского рода для слов «студент» и «преподаватель».
2. Автомат не предполагает отсутствие названия дисциплины на титульном листе.
3. Отсутствует обработка файлов.

Для масштабирования программы можно расширить регулярные выражения, добавив альтернативные варианты (например, "выполнил студент"/"выполнила студентка"); избавиться от привязки к конкретному университету, добавить альтернативные варианты (например, «Кафедра» вместо «Высшая школа»). Также в качестве масштабирования можно детализировать программный вывод об ошибках: конкретнее указывать, в какой строке титульного листа допущена ошибка.

Добавив модуль извлечения текста можно извлекать текстовое содержимое из PDF-файлов и передавать полученные строки на вход конечному автомату. Это позволит автоматизировать проверку больших массивов отчётов, например, при приёме лабораторных работ в электронной системе обучения.

Список литературы

- [1] Секция «Телематика» ВШТИИ. – [Электронный ресурс] (дата обращения: 01.04.2026).